

Project Documentation

Implement Client Script and UI Policy (Incident)

Date	1 October 2026
Team ID	SWTID-2026-8511
Project Name	Implement Client Script and UI Policy (Incident)

Note: this project is built on the ServiceNow platform, not the MERN stack. Each section of the documentation format is answered with the matching ServiceNow part.

1. Introduction

- **Project Title:** Implement Client Script and UI Policy (Incident)
- **Team Members:** [Add team member names and their roles]

2. Project Overview

- **Purpose:** Help service desk agents log complete and correct incidents by adding UI Policies and Client Scripts to the ServiceNow Incident form, so the form guides the agent while it is being filled instead of after it is saved.
- **Features:**
 - UI Policy shows or hides fields based on Category or State.
 - UI Policy makes fields mandatory, for example Resolution notes when State is Resolved.
 - UI Policy makes fields read-only when the incident is Closed.
 - Client Script (onChange) alerts the agent when a field such as Impact changes.
 - Client Script (onSubmit) validates the details and stops the submit with a clear message.

3. Architecture

- **Frontend:** The Incident form in the ServiceNow platform UI (browser). The UI Policy and Client Scripts run here, on the client side, and use the g_form API to read and change fields.
- **Backend:** Server-side logic on the ServiceNow platform: Business Rules and ACLs. They still apply when an incident is saved, even if the client-side rules are skipped.
- **Database:** The ServiceNow Incident table (incident), managed by the platform. The fields used by this project are listed below.

Field name	Label	Used by
number	Number	Identifies the incident
caller_id	Caller	Entered by the agent on the form
short_description	Short description	Checked by the onSubmit Client Script
category	Category	UI Policy shows or hides fields
impact / urgency / priority	Impact, Urgency, Priority	onChange Client Script and UI Policy conditions
state	State	UI Policy (mandatory and read-only conditions)
close_notes	Resolution notes	Made mandatory when State is Resolved
assignment_group	Assignment group	Resolver group that receives the incident

4. Setup Instructions

- **Prerequisites:**
 - A ServiceNow Developer account and a Personal Developer Instance (PDI).
 - The admin role on the instance.
 - A modern web browser (Chrome or Edge).
 - No Node.js, MongoDB or npm install is needed for this project.

- **Installation:**

1. Sign in to the ServiceNow Developer site and request a PDI.
2. Log in to the instance as admin.
3. Go to System Definition > UI Policies and create the UI Policies on the Incident table.
4. Go to System Definition > Client Scripts and create the onChange and onSubmit Client Scripts on the Incident table.
5. Set each rule to Active.
6. Optional: collect the rules in an Update Set to move them to another instance. No environment variables are needed.

5. Folder Structure

ServiceNow does not use project folders. The parts of the project live in the following places on the instance.

- **Client (browser side):**

- UI Policies: System Definition > UI Policies (table: Incident).
- Client Scripts: System Definition > Client Scripts (table: Incident).

- **Server (platform side):**

- Business Rules: System Definition > Business Rules.
- ACLs: System Security > Access Control.
- Update Set: System Update Sets > Local Update Sets (to package the work).

6. Running the Application

There is no npm start. The application runs on the ServiceNow instance as soon as the rules are active.

1. Log in to the instance and open Incident > Create New.
2. Fill the form and follow the test cases in section 10.
3. Watch the fields show, hide or become mandatory, and the messages appear.

7. API Documentation

This project does not add its own endpoints. The platform's standard Table API for the Incident table is available on the instance. Client Scripts and UI Policies do not run for these calls, so the server-side rules are the ones that apply.

Method	Endpoint	Purpose
GET	/api/now/table/incident?sysparm_limit=1	Read incidents
POST	/api/now/table/incident	Create an incident
PATCH	/api/now/table/incident/{sys_id}	Update an incident

Example request and response (values are examples):

```
POST /api/now/table/incident
{ "short_description": "Email not working", "caller_id": "<caller sys_id>" }

201 Created
{ "result": { "number": "INC0010001", "short_description": "Email not working",
              "state": "1", "sys_id": "<generated id>" } }
```

8. Authentication

- **Login:** Users sign in to the ServiceNow instance with a user name and password. The platform keeps the session.
- **Authorization:** Roles control access. Agents use the Incident form, and only users with the admin role can create or change UI Policies and Client Scripts. ACLs control access to records and fields.
- **API access:** Calls to the Table API use the same user accounts, with Basic authentication or OAuth. This project does not create its own tokens.

9. User Interface

The user interface is the standard ServiceNow Incident form. The project changes how the form behaves:

- Fields that are not needed are hidden, and needed fields are marked mandatory.
- Fields become read-only once the incident is Closed.
- A message appears when Impact is changed, and an error message appears when details are missing on submit.

10. Testing

Testing is manual. Each rule is tested on the Incident form with sample incidents, and form load time is checked with the browser developer tools.

Test	Scenario	Expected result
TC-1	Open a new incident	Form loads with the default fields and values
TC-2	Set State to Resolved with empty Resolution notes	Save is blocked; Resolution notes is mandatory
TC-3	Change Category	Relevant fields are shown and others are hidden
TC-4	Set State to Closed	Fields become read-only
TC-5	Change Impact to High	An alert message appears (onChange)
TC-6	Submit with Short description empty	Submit is stopped and a message shows the missing field (onSubmit)
TC-7	Submit a complete incident	Incident is saved successfully
TC-8	Open the Client Script list as a non-administrator	Creating or editing scripts is not allowed

11. Screenshots or Demo

Not included in this document.

12. Known Issues

- Client-side rules can be skipped by imports and API calls, so server-side checks are still needed.
- The scripts use fixed choice values (for example State 6 for Resolved), so they need an update if the choice values change.
- Too many Client Scripts on one form can slow it down.
- A free Personal Developer Instance can go to sleep when it is not used, so wake it before testing.

13. Future Enhancements

- Add Data Policies and Business Rules so the rules also apply to imports and integrations.
- Apply the same approach to the Problem and Change forms.
- Add Client Scripts to Service Portal and catalog forms.
- Build a dashboard that tracks reopened incidents before and after the rules.
- Use Predictive Intelligence to suggest Category and Assignment group.